

FRONTEND COHORT · VOLUME 02

CSS

the beginner's reference

• • •

A COMPANION GUIDE FOR

Tasty Bites — Web Development from Scratch

Everything you need for Lessons 03 & 04, plus modern CSS hacks and a full cheat sheet.

CONTENTS

What's inside

A complete tour of CSS for absolute beginners — the syntax, the color and font tools, the layout systems, and the modern features that make real websites feel polished.

01	What CSS actually is	<i>page 3</i>
02	Three ways to add CSS	<i>page 5</i>
03	Selectors — picking what to style	<i>page 6</i>
04	Colors	<i>page 9</i>
05	Typography	<i>page 11</i>
06	Sizes and units	<i>page 13</i>
07	The box model	<i>page 15</i>
08	CSS custom properties (variables)	<i>page 17</i>
09	Backgrounds and borders	<i>page 19</i>
10	The cascade and specificity	<i>page 20</i>
11	Display — block, inline, none	<i>page 21</i>
12	Flexbox — rows and columns	<i>page 22</i>
13	CSS Grid — true 2D layout	<i>page 25</i>
14	Positioning	<i>page 28</i>
15	Responsive design and media queries	<i>page 30</i>
16	Pseudo-classes and pseudo-elements	<i>page 32</i>
17	Transitions, animations, transforms	<i>page 34</i>
18	Modern CSS — clamp, color-mix, :has	<i>page 37</i>

19	Tips and little hacks	<i>page 39</i>
20	Common mistakes	<i>page 42</i>
21	The complete CSS cheat sheet	<i>page 43</i>
22	Where to go next	<i>page 46</i>

CHAPTER 01

What CSS actually is

If HTML is the skeleton, CSS is the skin, the clothes, the lighting, the makeup. Everything you see on a webpage that isn't structure comes from CSS.

CSS stands for **Cascading Style Sheets**. It's a separate language from HTML, with its own syntax, that exists to do one job: tell the browser how things should look. Colors, fonts, sizes, spacing, layout, animations — all CSS.

An HTML page with no CSS is fully functional. You can read every word. Every link works. But it'll look like a Word document from 1995 — Times New Roman, blue underlined links, no spacing. CSS is what turns that into a real-looking website.

HTML asks "what is it?" CSS asks "what should it look like?" Keep those two questions in different files and your life gets dramatically easier.

The shape of a CSS rule

```
h1 {  
  color: red;  
  font-size: 32px;  
}
```

Five things to spot:

- **Selector** — `h1` — which elements to style
- **Curly braces** — wrap the style rules
- **Property** — `color`, `font-size` — what to change
- **Value** — `red`, `32px` — what to change it to
- **Semicolon** — separates one declaration from the next. *End every declaration with one.*

How CSS gets applied

When the browser opens an HTML page, it reads the page first and builds the DOM. Then it reads the CSS, matches each rule to elements in the DOM, and works out the final appearance. Then it paints the page on screen.

This happens fast enough that you don't notice it. But the order matters when we discuss *the cascade* later — that's how the browser decides which rule wins when two rules say different things about the same element.

MENTAL MODEL

HTML is the noun. CSS is the adjective. JavaScript is the verb. *"Show a big red button that calls my friend."* Big and red = CSS. Button = HTML. Calls = JavaScript.

What CSS can't do

CSS is purely about appearance. It cannot:

- Change what text says (HTML's job)
- Calculate or store anything (JavaScript's job)
- Make a server request, save data, talk to an API (JavaScript)

What CSS *can* do is surprisingly broad: animations, hover effects, basic interactions like show/hide menus, even some "logic" via `:has()` and `:not()`. But anything dynamic — clicks that load data, forms that talk to a server — needs JavaScript.

— end of chapter one —

CHAPTER 02

Three ways to add CSS

Same effect, different setups. Use the first — it's how real websites are built.

1. External file — the right way

Create a separate file ending in `.css` (e.g. `styles.css`). Link it from the page's `<head>`:

```
<!-- in index.html -->
<head>
  <link rel="stylesheet" href="styles.css">
</head>
```

Why this wins:

- One CSS file can style many HTML pages
- The browser caches the file — visit page two, the CSS is already loaded
- Searching and editing styles is easy when they're all in one place

2. Internal — inside `<style>` in the page

```
<head>
  <style>
    h1 { color: red; }
  </style>
</head>
```

OK for tiny experiments or single-page demos. Not what you want for a real project — each page would need its own copy of the styles.

3. Inline — directly on the tag

```
<h1 style="color: red; font-size: 32px;">Hello</h1>
```

Almost never the answer. It's hard to maintain, impossible to reuse, and it wins over external CSS (which means it can break your design system in subtle ways). Avoid it.

THE ONE EXCEPTION

Inline styles are useful when JavaScript needs to set a value at runtime — for example, setting `style="--i: 5"` on cards so each one has its own animation delay. That's a legitimate use. Just don't write static color/font/spacing inline.

— end of chapter two —

CHAPTER 03

Selectors — picking what to style

A selector says "find these elements and apply these styles." There are dozens of selectors, but six cover 90% of what you'll ever write.

Element selector

Targets every tag of that type.

```
p { color: gray; }      /* every <p> on the page */
h1 { color: navy; }
img { width: 100%; }
```

Class selector

A **class** is a reusable label you put on a tag with `class="name"`. Target it with a leading dot.

```
/* HTML */
<p class="warning">Watch out!</p>

/* CSS */
.warning {
  color: red;
  font-weight: bold;
}
```

Classes are the workhorse of CSS. **Use them constantly.** A tag can have many classes:

```
<p class="warning big italic">...</p>
```

ID selector

An **id** is like a class but must be *unique on the page*. Target it with `#`.

```
/* HTML */
<header id="main-header">...</header>

/* CSS */
#main-header { background: black; }
```


Use IDs rarely — they have very high specificity (more on that in Chapter 10) and they don't reuse. Most of the time you want a class.

Descendant selector

Target elements *inside* other elements. Just put a space between the selectors.

```
article p { line-height: 1.6; }  
nav a { color: white; }  
.card .title { font-size: 1.5rem; }
```

"Every `<p>` inside an `<article>`." "Every `<a>` inside a `<nav>`." This is what makes CSS feel like real engineering.

Direct child selector

The `>` means "direct child only" — not grandchildren.

```
ul > li { color: blue; } /* top-level li only, not nested */
```

Multiple selectors

Apply the same rule to multiple selectors with a comma.

```
h1, h2, h3 {  
  font-family: 'Inter', sans-serif;  
}
```

Attribute selector

Target tags by their attribute values.

```
input[type="email"] { border-color: green; }  
a[href^="https://"] { color: blue; } /* starts with https:// */  
a[href$=".pdf"] { font-weight: bold; } /* ends with .pdf */
```

Universal selector

```
* { box-sizing: border-box; } /* every single element */
```

Used sparingly — usually only for one thing: setting `box-sizing: border-box` at the top of every stylesheet (you'll see why in the box-model chapter).

WHEN IN DOUBT, USE A CLASS

Classes are reusable, easy to add and remove, and have predictable specificity. They are the right answer 90% of the time. Reach for `id` only when you truly have a one-of-a-kind element.

— end of chapter three —

CHAPTER 04

Colors

CSS lets you write colors five different ways. They all produce the same color — choose based on readability and convenience.

The five ways

```
color: red;                /* named */
color: #ff0000;            /* hex */
color: #f00;               /* short hex */
color: rgb(255, 0, 0);     /* red/green/blue */
color: rgba(255, 0, 0, 0.5); /* + transparency */
color: hsl(0, 100%, 50%);  /* hue, saturation, lightness */
color: hsla(0, 100%, 50%, 0.5); /* + transparency */
```

Hex codes — the common one

Six hex digits: two for red, two for green, two for blue. `00` is none, `ff` is all. Online color pickers give you hex codes — that's what you'll paste 90% of the time.

RGB and HSL

Use `rgb()` or `hsl()` when you need to express a color in a form you can reason about. HSL is especially nice — change *lightness* to get a lighter or darker version of the same color without picking a brand-new hex.

Transparency

Add an `a` (alpha) value between 0 and 1:

```
background: rgba(0, 0, 0, 0.5); /* half-transparent black */
```

Common color properties

```
color: #2B1D13;           /* text color */
background-color: #FAF6EF;
border-color: #E4D9C5;
outline-color: #C2563A;
caret-color: #C2563A;     /* the text cursor */
```

The Tasty Bites palette

The recipe project uses these five swatches — a warm food-magazine feel. Steal them.



Cream #FAF6EF · page background



Paper #F3ECD9 · card surface



Espresso #2B1D13 · text



Terracotta #C2563A · accent



Sage #7A8A6B · secondary



Mustard #D4A24C · highlight

HACK — NEVER USE PURE BLACK

Pure black (**#000**) looks harsh on screens. Use a very dark warm or cool color instead — like **#2B1D13** (espresso) or **#1F2937** (slate). Subtle, but every magazine and high-end site does this.

— end of chapter four —

CHAPTER 05

Typography

Fonts shape how a page feels more than any other CSS choice. Spend time here — it pays off forever.

The properties you'll set most

```
font-family: 'Inter', sans-serif;
font-size: 1.1rem;
font-weight: 600;
font-style: italic;
line-height: 1.6;
letter-spacing: 0.02em;
text-align: left;           /* or center, right, justify */
text-transform: uppercase; /* or lowercase, capitalize */
text-decoration: underline; /* or none, line-through */
```

font-family — the typeface

Give a comma-separated list. The browser tries each one until it finds one installed. Always end with a generic family — `sans-serif`, `serif`, `monospace` — so you have a guaranteed fallback.

```
font-family: 'DM Serif Display', Georgia, serif;
```

Loading real fonts from Google Fonts

System fonts (Georgia, Arial, Helvetica) work, but every modern site uses custom fonts. The easiest source: **Google Fonts** — free, fast, and reliable.

```
<!-- in <head> -->
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link rel="stylesheet" href="https://fonts.googleapis.com/css2?
family=Inter:wght@400;600;700&display=swap">
```

Then use the font in CSS:

```
body { font-family: 'Inter', sans-serif; }
```

The `&display=swap` part tells the browser: "show the fallback font right away; swap in the real one when it loads." Without it, text can be invisible while the font downloads.

font-weight — bold and light

```
font-weight: normal;    /* = 400 */
font-weight: bold;      /* = 700 */
font-weight: 300;       /* light */
font-weight: 900;       /* black */
```

Weights are numbers 100 to 900 in steps of 100. The font you load has to include the weight you ask for — that's why the Google Fonts URL lists weights like `wght@400;600;700`.

line-height — vertical breathing room

The space between lines of text. Use a **unitless number** (the multiplier of the font size).

```
line-height: 1.5;    /* good for body text */
line-height: 1.2;    /* tight, for big headings */
line-height: 1.8;    /* airy, for long-form reading */
```

Editorial type pairings

Pairing a serif headline with a sans-serif body is the magazine classic. The course uses:

ROLE	FONT	WHY
Headlines	DM Serif Display	Warm, editorial weight, thick/thin contrast
Body	Inter	Designed for screens; tall x-height, readable
Italic accent	Cormorant Garamond italic	Refined italic for bylines and pull quotes

HACK — TEXT-WRAP: BALANCE

On headlines, add `text-wrap: balance`. The browser balances line lengths so you don't get one widow word on the last line. Stunning, free, modern.

```
h1 { text-wrap: balance; }
```

— end of chapter five —

CHAPTER 06

Sizes and units

Pick the right unit and your layout breathes. Pick the wrong one and it looks weird at any other screen size.

Absolute units

UNIT	WHAT IT IS	WHEN TO USE
<code>px</code>	Pixels — a fixed dot	Borders, shadows, small details
<code>in</code> , <code>cm</code> , <code>mm</code>	Real-world units	Print stylesheets only

Relative units (use these for most things)

UNIT	RELATIVE TO	WHEN TO USE
<code>%</code>	Parent's size	Widths inside containers
<code>em</code>	The element's own font size	Padding/margin on a component
<code>rem</code>	Root font size (<code><html></code> , usually 16px)	Most spacing, sizing, type
<code>vw</code>	1% of viewport width	Hero text, full-bleed sections
<code>vh</code>	1% of viewport height	Full-screen heros
<code>ch</code>	Width of "0" in the font	Max line widths for readability
<code>fr</code>	Fraction of grid free space	CSS Grid column sizing

The rem rule of thumb

Use `rem` for almost everything. It scales with the user's browser font size, so users who set their browser to a larger size (eyesight, accessibility) get a larger version of your whole site — for free.

```
/* Better */
.card { padding: 1.5rem; }

/* Brittle */
.card { padding: 24px; }
```

clamp() — fluid sizes between two limits

This one function changed how modern sites handle responsive typography. Three arguments: minimum, ideal, maximum.

```
font-size: clamp(1rem, 0.5vw + 0.9rem, 1.25rem);  
font-size: clamp(2rem, 4vw + 1rem, 4.5rem); /* hero headline */
```

The browser smoothly interpolates between the minimum (small screens) and the maximum (huge screens). One rule replaces three media queries.

Width and height

```
width: 300px; /* fixed width */  
width: 100%; /* fill the parent */  
max-width: 720px; /* cap the width, allow smaller */  
min-height: 100vh; /* at least one screen tall */  
aspect-ratio: 16 / 9; /* lock the shape */
```

HACK — PREFER MAX-WIDTH

Almost always use `max-width` instead of `width`. `max-width` says "no wider than this, but smaller is fine" — which is what you want on every screen size.

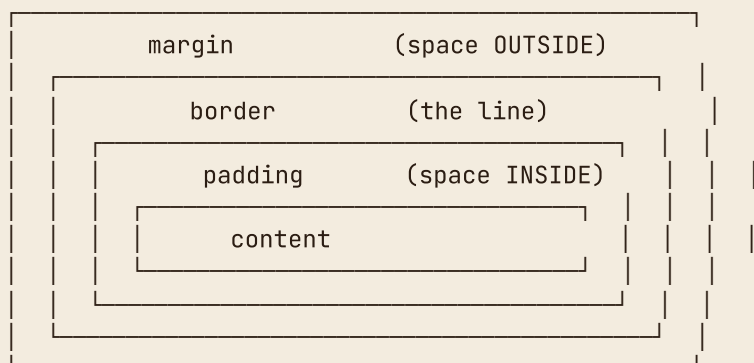
— end of chapter six —

CHAPTER 07

The box model

Every HTML element is a rectangle made of four layers. Once you really understand this, CSS gets dramatically easier.

The four layers, from inside out



- **Content** — the actual text or image
- **Padding** — space between content and border (inside the box)
- **Border** — the line that surrounds the box
- **Margin** — space between the box and other boxes (outside)

Setting them

```
.card {  
  padding: 16px;  
  border: 2px solid black;  
  margin: 20px;  
}
```

Shorthand — set all four sides or each one

```
margin: 10px;                /* all 4 sides */  
margin: 10px 20px;          /* top/bottom, left/right */  
margin: 10px 20px 5px 0;    /* top, right, bottom, left */  
  
margin-top: 10px;           /* left + right at once */  
margin-inline: auto;        /* top + bottom at once */  
margin-block: 2rem;
```

`margin-inline: auto` is a beautiful trick for horizontally centering a block element. Same as `margin: 0 auto` but more explicit.

The one rule that fixes everything

By default, `width` means the width of the content only. Padding and border are *added on top*. This trips up beginners: set a card to `width: 300px; padding: 20px;` and the actual rendered width is 340px.

Fix: tell the browser to include padding and border in the width calculation. Put this at the top of every stylesheet:

```
* { box-sizing: border-box; }
```

The `*` means "every element." This single line removes an entire category of frustration. Always include it.

MARGIN COLLAPSING — THE ONE WEIRD QUIRK

When two vertical margins meet (one block's bottom against another's top), they don't add — they collapse to the larger of the two. So a 20px bottom margin + 30px top margin = 30px total, not 50px. Doesn't happen with flex or grid containers. Worth knowing about.

Inline vs block — only block elements obey margin/padding properly

By default, block elements (`div` , `p` , `h1`) take a full line and respect width/height. Inline elements (`span` , `a` , `strong`) flow within text and ignore top/bottom margin. To make an inline element accept margin and width, set `display: inline-block` .

— end of chapter seven —

CHAPTER 08

CSS custom properties

"Variables for CSS." Define a value once, reuse it everywhere. Change it once, every use updates.

How they work

```
/* Define on :root so they're available everywhere */
:root {
  --color-accent: #C2563A;
  --gap: 1.5rem;
}

/* Use with var() */
a { color: var(--color-accent); }
.grid { gap: var(--gap); }
```

Two rules:

- Custom property names start with `--`
- You retrieve them with `var(--name)`

Why they're transformative

Imagine 60 different places use `#C2563A` in your stylesheet. Now the client wants the brand color to be slightly more orange. Without variables: find & replace 60 instances, hope you don't miss one. With variables: change one line, done.

A real palette

```
:root {
  /* Colors */
  --cream:      #FAF6EF;
  --paper:      #F3ECDF;
  --espresso:   #2B1D13;
  --terracotta: #C2563A;
  --sage:       #7A8A6B;
  --rule:       #E4D9C5;

  /* Fonts */
  --font-display: 'DM Serif Display', serif;
  --font-body:    'Inter', sans-serif;
  --font-accent:  'Cormorant Garamond', serif;

  /* Spacing scale */
  --space-sm: 0.5rem;
  --space:    1rem;
  --space-lg: 2rem;
}
```

This *is* your design system. Every color, font, and spacing decision in one block at the top. The rest of the stylesheet just uses `var()`.

Default values with `var()`

You can pass a fallback as a second argument — used if the variable isn't defined.

```
color: var(--my-color, #333); /* falls back to #333 */
```

Variables can be set on any element — and inherit

You don't have to put them on `:root`. Set them on a card and they cascade down to its children. This is how you build a staggered animation:

```
/* HTML: <article style="--i: 2">...</article> */

.card {
  animation: fade-up 0.6s ease forwards;
  animation-delay: calc(var(--i, 0) * 80ms);
}
```

Card 0 starts at 0ms, card 1 at 80ms, card 2 at 160ms. Without variables this kind of per-element customization is painful.

HACK — SEMANTIC NAMES BEAT LITERAL ONES

Don't write `--red: #C2563A` — write `--color-accent: #C2563A` . If the brand color later becomes blue, the literal name lies. Semantic names age well.

— end of chapter eight —

CHAPTER 09

Backgrounds and borders

Two surfaces that quietly make a card feel like a card.

Background properties

```
background-color: #FAF6EF;
background-image: url('hero.jpg');
background-size: cover;          /* fill the box, crop if needed */
background-position: center;
background-repeat: no-repeat;
background-attachment: fixed;    /* don't scroll with the page */

/* Shorthand */
background: #FAF6EF url(hero.jpg) center / cover no-repeat;
```

Gradients — built into the background

```
/* Linear */
background: linear-gradient(to right, #FAF6EF, #F3ECDf);
background: linear-gradient(135deg, #C2563A, #D4A24C);

/* Radial — circular tint */
background: radial-gradient(circle at top, #D4A24C 0%, transparent 60%);

/* Conic — pie slices */
background: conic-gradient(#C2563A, #D4A24C, #7A8A6B, #C2563A);
```

Borders

```
border: 1px solid #E4D9C5;
border-radius: 12px;          /* rounded corners */
border-radius: 999px;         /* fully rounded pill */
border-radius: 12px 12px 0 0; /* top corners only */

border-top: 2px solid var(--terracotta);
border-bottom: 1px dashed var(--rule);
```

Border styles: `solid`, `dashed`, `dotted`, `double`, `none`.

Shadows — depth without weight

```
box-shadow: 0 4px 8px rgba(0, 0, 0, 0.1);
box-shadow: 0 12px 24px -12px rgb(43 29 19 / 0.12);    /* soft lift */

/* Layered shadows for realism */
box-shadow:
  0 1px 2px rgb(43 29 19 / 0.04),
  0 12px 24px -12px rgb(43 29 19 / 0.12);

text-shadow: 2px 2px 4px rgba(0,0,0,0.3);    /* text only */
```

Shadow values: `X Y blur spread color` . Negative X moves left, negative Y moves up.

HACK — NEVER USE PURE BLACK SHADOWS

Tint shadows with your text color, dimmed way down. A shadow of `rgb(43 29 19 / 0.12)` (12% espresso) feels warm and integrated. Plain `rgba(0,0,0,0.1)` reads grey and dead. Magazine sites do this universally.

— end of chapter nine —

CHAPTER 10

The cascade and specificity

When two rules conflict, the browser has to pick a winner. This is the only "logic" CSS does — and it's worth understanding.

When two rules apply

```
p { color: blue; }  
.special { color: red; }
```

```
<p class="special">What color am I?</p>
```

Answer: red. The class selector wins over the tag selector.

The specificity ranking (rough)

1. **Inline** (`style="..."`) beats everything
2. **IDs** (`#header`) beat classes
3. **Classes** (`.card`) beat element names
4. **Element names** (`p`) beat the universal selector

When two rules are equally specific, **the one written later wins**. That's why the order of your stylesheet matters: later rules override earlier ones.

Combined selectors raise specificity

```
.card .title { color: red; }      /* two classes */  
.title { color: blue; }          /* one class */  
  
/* Inside a .card, .title becomes red.  
   The first rule has higher specificity. */
```

Inheritance — kids get parents' values (some properties only)

Some properties — `color`, `font-family`, `line-height`, `text-align` — pass down to children automatically. Set `color` on `body` and every child inherits it (unless they override it).

Properties like `margin` , `padding` , `border` , and `background` do *not* inherit.

The DevTools rescue

Right-click any element in the browser → **Inspect**. In the side panel you'll see every CSS rule that applies, in order, with the winning ones highlighted. When something looks wrong, open DevTools first — it tells you why.

DON'T REACH FOR !IMPORTANT

You can override the cascade with `!important` at the end of any value. Don't. It cascades poorly and breaks your design system. If you need `!important` , your specificity is wrong — fix that instead.

— end of chapter ten —

CHAPTER 11

Display — block, inline, none

Every element has a default display type. Knowing the type tells you how it'll behave.

The four basic types

VALUE	BEHAVIOR	DEFAULTS FOR
<code>block</code>	Takes full width, starts on a new line	<code>div</code> , <code>p</code> , <code>h1</code> , <code>section</code>
<code>inline</code>	Only as wide as content, flows in text	<code>span</code> , <code>a</code> , <code>strong</code> , <code>em</code>
<code>inline-block</code>	Flows like inline, accepts width/padding like block	(set manually)
<code>none</code>	Removes the element entirely (no space taken)	(set manually)

Toggling visibility

```
display: none;           /* removed — no space */
visibility: hidden;      /* invisible — space preserved */
opacity: 0;              /* see-through but still there */
```

The big two: flex and grid

Two display values do most of the layout work in modern CSS. They each get their own chapter next.

```
.row { display: flex; }    /* 1D layout — row or column */
.grid { display: grid; }   /* 2D layout — rows AND columns */
```

— end of chapter eleven —

CHAPTER 12

Flexbox — rows and columns

A layout system for one direction at a time — a row OR a column. Most navbar/header/toolbar/footer layouts are Flexbox.

The basic move

```
/* HTML */
<div class="row">
  <div>A</div>
  <div>B</div>
  <div>C</div>
</div>

/* CSS */
.row {
  display: flex;
  gap: 1rem;      /* space between children */
}
```

A

B

C

Three children laid out in a row by their parent.

Container properties — set on the parent

flex-direction

```
flex-direction: row;           /* default — left to right */
flex-direction: column;        /* top to bottom */
flex-direction: row-reverse;    /* right to left */
```

justify-content — alignment along the main axis

```
justify-content: flex-start;    /* default */
justify-content: center;
justify-content: flex-end;
justify-content: space-between; /* edges + equal gaps */
justify-content: space-around; /* equal space around each */
justify-content: space-evenly;
```

align-items — alignment along the cross axis

```
align-items: stretch;          /* default — fill height */
align-items: center;            /* vertically center (rows) */
align-items: flex-start;
align-items: flex-end;
align-items: baseline;
```

gap, flex-wrap

```
gap: 1rem;                      /* space between children */
flex-wrap: wrap;                /* wrap to next line if too wide */
```

Item properties — set on a child

```
flex: 1;                        /* grow to fill leftover space (equal share) */
flex: 2;                        /* twice the share of siblings with flex: 1 */
flex: 0 0 200px;               /* don't grow, don't shrink, be 200px wide */
align-self: flex-end;          /* override align-items for this child only */
order: 2;                      /* reorder visually without changing HTML */
```

The classic header layout

```
/* Logo left, nav right, vertically centered */
header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
}
```

Centering — finally easy

The legendary "how do I center a div" question. With Flexbox:

```
.parent {  
  display: flex;  
  justify-content: center; /* horizontal */  
  align-items: center; /* vertical */  
  min-height: 100vh;  
}
```

HACK — USE GAP INSTEAD OF MARGINS

Old Flexbox tutorials use `margin-right` on every child except the last. That's outdated.

`gap` works in Flexbox and Grid and handles the last item automatically. Always use `gap`.

— end of chapter twelve —

CHAPTER 13

CSS Grid — true 2D layout

Flexbox handles a row or a column. Grid handles both at once — a real grid of rows and columns. Card galleries, page layouts, magazine spreads.

The basic move

```
.gallery {  
  display: grid;  
  grid-template-columns: 1fr 1fr 1fr; /* 3 equal columns */  
  gap: 1.5rem;  
}
```

`1fr` means "one fraction of the available space." Three `1fr` = three equal columns.

Mix fixed and flexible

```
grid-template-columns: 200px 1fr; /* sidebar + main */  
grid-template-columns: 1fr 2fr 1fr; /* middle is 2x */  
grid-template-columns: repeat(4, 1fr); /* shortcut: 4 equal */
```

The killer feature: auto-fit + minmax

You rarely want to hardcode "3 columns." You want "as many columns as fit, each at least 18rem." That's this:

```
grid-template-columns: repeat(auto-fill, minmax(min(100%, 18rem), 1fr));
```

What this says:

- **repeat(auto-fill, ...)** — make as many columns as fit
- **minmax(18rem, 1fr)** — each column is at least 18rem, but stretches to fill space
- **min(100%, 18rem)** — on a very narrow screen, drop to one full-width column instead of horizontal scroll

Memorize this snippet. It's the responsive card grid every modern site uses.

Featured

B

C

D

Featured card spans 2 columns. Mobile? It collapses to one.

Featured cards — items can span columns

```
.featured {
  grid-column: span 2;      /* takes 2 columns */
}

.tall {
  grid-row: span 2;        /* takes 2 rows */
}

.exact {
  grid-column: 2 / 4;      /* from line 2 to line 4 */
}
```

Grid rows — let the content decide

You almost never need to set rows explicitly — Grid creates them as your items wrap. If you do need to:

```
grid-template-rows: 100px 1fr 50px;
grid-auto-rows: 200px;      /* every auto-created row */
```

Named grid areas — for whole-page layout

```
.layout {
  display: grid;
  grid-template-columns: 200px 1fr;
  grid-template-rows: auto 1fr auto;
  grid-template-areas:
    "header header"
    "sidebar main"
    "footer footer";
}

header { grid-area: header; }
.sidebar { grid-area: sidebar; }
main { grid-area: main; }
footer { grid-area: footer; }
```

The ASCII-art layout is literal CSS. Beautiful, readable, and self-documenting.

When to use Flexbox vs Grid

USE	FLEXBOX	GRID
Header with logo + nav	✓	
Navigation toolbar	✓	
Recipe card gallery		✓
Sidebar + main + footer layout		✓
Form labels + inputs in a row	✓	
Image grid with featured + tiles		✓

— end of chapter thirteen —

CHAPTER 14

Positioning

Move an element out of the normal flow — to overlay it, stick it to the top, or pin it to the corner.

The five values

VALUE	WHAT IT DOES
<code>static</code>	Default — flows in document order
<code>relative</code>	Like static but offset by <code>top</code> / <code>left</code> ; siblings ignore the shift
<code>absolute</code>	Positioned relative to the nearest positioned ancestor
<code>fixed</code>	Pinned to the viewport — stays visible while scrolling
<code>sticky</code>	Acts relative until a threshold, then sticks

`relative` — offset without disrupting siblings

```
.badge {  
  position: relative;  
  top: -5px;      /* nudge up 5px */  
}
```

Useful as a container for absolutely-positioned children — see below.

`absolute` — float over the page

```
.card { position: relative; }  
  
.card .badge {  
  position: absolute;  
  top: 1rem;  
  right: 1rem;  
}
```

The badge is removed from normal flow and pinned to the top-right corner of the nearest *positioned* ancestor (the `.card`). Perfect for "Featured," "New," "Sale" tags on a card.

fixed — pinned to the screen

```
.fab { /* a "floating action button" */  
  position: fixed;  
  bottom: 2rem;  
  right: 2rem;  
}
```

Stays put while the page scrolls. Common for chat bubbles, "back to top" buttons, sticky cookie banners.

sticky — the magic one

```
.section-title {  
  position: sticky;  
  top: 0;  
  background: var(--cream);  
}
```

Scrolls normally until it hits the top, then sticks. Used for sticky headers, sticky table-of-contents sidebars. Underrated.

z-index — what's in front

When elements overlap, `z-index` controls which appears on top. Higher = in front. Only works on positioned elements.

```
z-index: 10;
```

HACK — PIN BADGES WITH ABSOLUTE

Almost every "Featured", "New", "Sale" badge you see on a card is `position: absolute` on a `position: relative` card. Pattern worth memorizing.

— end of chapter fourteen —

CHAPTER 15

Responsive design

A site that looks good on a 6-inch phone AND a 27-inch desktop. The key tools: fluid units, media queries, and modern auto-collapsing layouts.

Step one: stop fighting

Modern CSS does a lot of responsive work automatically. Some changes you can make to set yourself up:

- Use `max-width` instead of `width`. Let content shrink.
- Use `%`, `rem`, `clamp()` instead of fixed pixels.
- Use the modern grid recipe (Chapter 13) — it auto-reflows.
- Always include the viewport meta tag in your HTML.

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Media queries — different CSS at different sizes

```
.card { font-size: 1rem; }

/* When the screen is 768px wide or LESS */
@media (max-width: 768px) {
  .card { font-size: 0.875rem; }
}

/* When the screen is 1024px wide or MORE */
@media (min-width: 1024px) {
  .card { font-size: 1.125rem; }
}
```

Common breakpoints

SIZE	ROUGHLY MEANS
up to 640px	phone
up to 768px	large phone / small tablet
up to 1024px	tablet
over 1024px	laptop and desktop

You don't need many. One or two breakpoints per design carries most sites.

Mobile-first vs desktop-first

Mobile-first (recommended): write the small-screen styles first, then use `min-width` media queries to add complexity for bigger screens.

```
.card { padding: 1rem; }  
  
@media (min-width: 768px) {  
  .card { padding: 2rem; }  
}
```

Mobile is simpler. Adding more for big screens is more natural than removing for small ones.

Test responsively

In Chrome / Edge, right-click → **Inspect** → click the phone/tablet icon in DevTools. You can resize the viewport or pick a device preset. Always test at 375px (a phone) and the size you usually develop at.

The "no media queries needed" trick

Combined with the modern grid recipe, you'll often handle responsiveness entirely through CSS Grid + `clamp()` without a single media query:

```
.grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(min(100%, 18rem), 1fr));  
  gap: clamp(1rem, 2vw, 2rem);  
}  
  
h1 {  
  font-size: clamp(2rem, 4vw + 1rem, 4.5rem);  
}
```

The grid auto-collapses, the type auto-scales. Less CSS, more elegance.

— end of chapter fifteen —

CHAPTER 16

Pseudo-classes and pseudo-elements

Style elements based on state (hovered, focused, checked) or pieces of an element (first letter, before/after).

Pseudo-classes — single colon

State the element is in.

SELECTOR	WHEN IT MATCHES
<code>:hover</code>	Mouse is over the element
<code>:focus</code>	The element has keyboard focus
<code>:active</code>	Being clicked right now
<code>:visited</code>	Link has been visited
<code>:disabled</code>	Form field is disabled
<code>:checked</code>	Checkbox or radio is on
<code>:first-child</code>	First child of its parent
<code>:last-child</code>	Last child of its parent
<code>:nth-child(n)</code>	The nth child (e.g. <code>:nth-child(odd)</code>)
<code>:not(.x)</code>	Everything that isn't <code>.x</code>
<code>:empty</code>	Element with no children
<code>:placeholder-shown</code>	Input is still showing its placeholder

Hover effects — the bread and butter

```
button {  
  background: var(--terracotta);  
  color: white;  
  cursor: pointer;  
  transition: background 0.2s ease;  
}  
  
button:hover {  
  background: #A23C2A;    /* darker */  
}
```

The `transition` property makes the hover change *glide* instead of snap. You'll see it covered in Chapter 17.

Focus styles — please don't kill them

Browsers show a focus ring when you tab to a focusable element. Don't remove it without replacing it — keyboard users rely on it.

```
input:focus {  
  outline: none;  
  border-color: var(--terracotta);  
  box-shadow: 0 0 0 4px color-mix(in srgb, var(--terracotta) 15%, transparent);  
}
```

:nth-child — striped rows, every other thing

```
tr:nth-child(even) { background: var(--paper); }  
.card:nth-child(3n) { color: var(--sage); }    /* every 3rd */
```

Pseudo-elements — double colon

Style a part of an element that isn't a real element.

SELECTOR	WHAT IT TARGETS
<code>::before</code>	An invisible element inserted before the content
<code>::after</code>	An invisible element inserted after the content
<code>::first-letter</code>	The first letter (drop caps!)
<code>::first-line</code>	The first line of text
<code>::placeholder</code>	The placeholder text in an input
<code>::selection</code>	Text the user has highlighted

Drop caps

```
p.lede::first-letter {  
  float: left;  
  font-family: var(--font-display);  
  font-size: 3.2em;  
  line-height: 0.85;  
  padding: 0.05em 0.15em 0 0;  
  color: var(--terracotta);  
}
```

Decorative `::before` / `::after`

```
.eyebrow::before {  
  content: "— ";  
  color: var(--terracotta);  
}
```

The `content` property is required — even if it's empty (`content: ""`). The element doesn't exist without it.

— end of chapter sixteen —

CHAPTER 17

Transitions, animations, transforms

Movement makes a site feel alive. Used sparingly, it's premium. Used too much, it's nauseating.

transition — animate property changes

```
button {  
  background: var(--terracotta);  
  transition: background 0.2s ease;  
}  
  
button:hover {  
  background: #A23C2A;  
}
```

Format: `property duration timing-function delay`.

Animate multiple properties

```
transition: background 0.2s ease, transform 0.3s ease;  
transition: all 0.2s ease; /* every animatable property */
```

Timing functions

How the speed curves over time.

FUNCTION	FEEL
<code>linear</code>	Robotic, even speed
<code>ease</code>	Default — gentle in and out
<code>ease-in</code>	Slow start, fast end
<code>ease-out</code>	Fast start, slow end
<code>cubic-bezier(.2, .8, .2, 1)</code>	Custom — the snappy magazine feel

transform — move, rotate, scale

```
transform: translateY(-4px);           /* nudge up */
transform: translateX(10px);
transform: scale(1.05);                /* gentle zoom */
transform: rotate(-3deg);
transform: scale(1.05) rotate(-2deg); /* combine */
```

Transforms are fast — the browser uses the GPU. Always animate `transform` instead of `top` / `left` for movement.

The card lift on hover

```
.card {
  transition: transform 0.35s cubic-bezier(.2, .8, .2, 1),
              box-shadow 0.35s ease;
}

.card:hover {
  transform: translateY(-4px);
  box-shadow: 0 24px 48px -16px rgb(43 29 19 / 0.22);
}
```

Image zoom inside a wrapper

```
.card-image-wrap { overflow: hidden; }

.card-image-wrap img {
  transition: transform 0.5s ease;
}

.card:hover .card-image-wrap img {
  transform: scale(1.05);
}
```

The image grows; the wrapper crops the overflow. Card feels alive without dancing around.

@keyframes — full animations

For things bigger than a hover state. Define a sequence, then apply it.

```
@keyframes fade-up {
  from {
    opacity: 0;
    transform: translateY(12px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.card {
  opacity: 0;
  animation: fade-up 0.6s ease forwards;
  animation-delay: calc(var(--i, 0) * 80ms);
}
```

forwards means "keep the final state after the animation ends." Without it the element snaps back to its start.

Respect reduced motion

Some people get motion-sick from animation. Browsers expose their preference. Honor it:

```
@media (prefers-reduced-motion: reduce) {
  * {
    animation-duration: 0.001ms !important;
    transition-duration: 0.001ms !important;
  }
}
```

HACK — USE `CUBIC-BEZIER(.2, .8, .2, 1)`

This particular curve has a snappy start and a soft landing. It's what most premium product sites use. Replace your `ease` with it and movements feel more expensive.

— end of chapter seventeen —

CHAPTER 18

Modern CSS

Features that landed in browsers in the last two years. Each one replaces a workaround that used to take ten lines of code.

aspect-ratio — lock an element's shape

```
img {  
  width: 100%;  
  aspect-ratio: 4 / 5;      /* portrait card */  
  object-fit: cover;      /* crop to fill */  
}
```

Every card image now has the same shape regardless of source dimensions. No more uneven grids.

color-mix() — blend colors on the fly

```
background: color-mix(in srgb, var(--terracotta) 12%, transparent);  
background: color-mix(in srgb, var(--terracotta), white 30%);
```

"12% terracotta on transparent" gives a soft tinted pill. Change the brand color and every tinted variant updates automatically. No more `--terracotta-100`, `--terracotta-200`, etc.

:is() — group selectors

```
/* Old - repetitive */  
article h1, article h2, article h3 { margin-top: 1.5rem; }  
  
/* Modern - :is() */  
article :is(h1, h2, h3) { margin-top: 1.5rem; }
```

:where() — same, but with zero specificity

```
:where(article, section) p { line-height: 1.6; }
```

Useful for resets and base styles you want to be easy to override.

:has() — parent selector

Style an element based on its children. This was impossible in CSS until 2023.

```
/* Style the grid only when it contains an empty-state message */
.recipe-grid:has(.empty) {
  display: block;
  text-align: center;
  padding: 3rem 0;
}

/* Cards with a "new" badge get a special border */
.card:has(.badge--new) {
  outline: 2px solid var(--terracotta);
}
```

text-wrap: balance

```
h1, h2, h3 { text-wrap: balance; }
```

The browser balances line lengths across a heading. No more single-word last lines on hero text.

CSS Nesting

Write child rules inside parent rules — just like SASS, but native:

```
.card {
  background: white;

  h3 {
    color: var(--terracotta);
  }

  &:hover { /* & refers to the parent (.card) */
    transform: translateY(-4px);
  }
}
```

Container queries — bonus feature

Like media queries, but they respond to the size of a *parent container*, not the viewport. Powerful for component-driven design.

```
.card-wrap { container-type: inline-size; }  
  
@container (min-width: 400px) {  
  .card { font-size: 1.2rem; }  
}
```

— end of chapter eighteen —

CHAPTER 19

Tips and little hacks

Steal these. They're the difference between code that works and code that feels professional.

Open DevTools and live

Right-click → Inspect. Edit any property live. See every rule that applies. Add classes to elements without touching the file. Then copy the working version back. The browser is the best CSS playground.

Use the universal box-sizing rule

```
* { box-sizing: border-box; }
```

First line of every stylesheet you'll ever write. Makes width include padding + border. Removes a whole class of bugs.

Reset the body

```
body {  
  margin: 0;  
  font-family: var(--font-body);  
  color: var(--espresso);  
  line-height: 1.6;  
}
```

Browsers add 8px of margin to `body` by default. Always zero it.

Center horizontally with `margin: 0 auto`

```
.container {  
  max-width: 720px;  
  margin: 0 auto;      /* horizontal: auto = center */  
}
```

Fully rounded pills

```
border-radius: 999px;
```

For buttons, tags, search bars. Any big number — the browser caps at 50% — so `999px` works for any height.

Limit reading width

```
article { max-width: 65ch; }
```

`ch` = width of one character. 60-75ch is the readability sweet spot for prose. Magazine sites swear by it.

cursor: pointer on interactive things

```
button, [role="button"] { cursor: pointer; }
```

Tells the user "this is clickable." Free UX win.

Inherit instead of repeating

```
button, input {  
  font-family: inherit;  
  font-size: inherit;  
}
```

By default, form elements ignore the page font. Force them to inherit.

Truncate long text with ellipsis

```
.truncate {  
  white-space: nowrap;  
  overflow: hidden;  
  text-overflow: ellipsis;  
}
```

Two-line truncation

```
.clamp-2 {  
  display: -webkit-box;  
  -webkit-line-clamp: 2;  
  -webkit-box-orient: vertical;  
  overflow: hidden;  
}
```


Style placeholder text

```
input::placeholder {  
  color: rgb(43 29 19 / 0.4);  
  font-style: italic;  
}
```

Hide visually, keep for screen readers

```
.sr-only {  
  position: absolute;  
  width: 1px; height: 1px;  
  padding: 0; margin: -1px;  
  overflow: hidden;  
  clip: rect(0, 0, 0, 0);  
  white-space: nowrap;  
  border: 0;  
}
```

Use for icon-button labels: `<span class="sr-only">Search</span>` next to a magnifying-glass SVG.

The "box-shadow stack" for realistic depth

```
box-shadow:  
  0 1px 2px rgb(0 0 0 / 0.04),  
  0 4px 8px rgb(0 0 0 / 0.06),  
  0 16px 32px -8px rgb(0 0 0 / 0.12);
```

Three layered shadows feel realistic. One big shadow feels cheap.

Backdrop blur — glassmorphism

```
.glass {  
  background: rgba(255, 255, 255, 0.6);  
  backdrop-filter: blur(12px);  
}
```

Smooth scroll on anchor links

```
html { scroll-behavior: smooth; }
```

Use logical properties for international layouts

```
margin-inline: auto;           /* left + right */
padding-block: 1rem;           /* top + bottom */
border-inline-start: ...;      /* left in English, right in Arabic */
```

Group fonts and font-weights in one Google Fonts URL

```
https://fonts.googleapis.com/css2?
  family=DM+Serif+Display
  &family=Inter:wght@400;500;700
  &family=Cormorant+Garamond:ital@1
  &display=swap
```

One `<link>`, multiple fonts. Faster to load.

Use aspect-ratio + object-fit together

Always pair them on images. Otherwise a wide source image will stretch the card.

```
img {
  width: 100%;
  aspect-ratio: 4 / 5;
  object-fit: cover;
}
```

Hover only on devices with a real pointer

```
@media (hover: hover) {
  .card:hover { transform: translateY(-4px); }
}
```

Phones don't really hover. This prevents the hover state from sticking on touch.

— end of chapter nineteen —

CHAPTER 20

Common mistakes

Every beginner runs into these. Knowing them by name makes them quick to fix.

1. Missing semicolons

Every declaration ends with `;`. Miss one and the next line silently fails.

2. Forgetting box-sizing: border-box

Without it, padding adds to width. Cards overflow. Layouts break. Add the universal rule.

3. Centering with text-align: center on a block

`text-align` centers text, not the element itself. To center a block: `margin: 0 auto` (with a width) or wrap it in a flex parent.

4. Using width: 100% when you mean max-width

`width: 100%` fights the parent's padding. `max-width` respects it.

5. Pixel-perfect everything

Designing only for the size of your monitor. Open DevTools, switch to mobile view, develop responsively from the start.

6. Too many breakpoints

You don't need different styles for every device. One or two breakpoints handles most designs.

7. !important as a fix

It papers over a specificity problem instead of solving it. Almost always wrong.

8. Removing focus outlines without a replacement

`outline: none` with nothing in its place breaks keyboard navigation. Always replace, never just remove.

9. Forgetting cursor: pointer on custom buttons

Real `<button>` elements get it for free. Custom ones made from `div` don't.

10. Overusing absolute positioning

Reaching for `position: absolute` to lay things out instead of Flexbox / Grid is a beginner trap. Use absolute only for things truly out of flow — badges, overlays, popovers.

11. Fighting the cascade with very specific selectors

If you find yourself writing `article .card .body p span.note`, take a step back. Add a class to the span instead.

12. Forgetting `:hover` needs a transition

Without `transition`, hover effects snap on and off harshly. Add a transition every time.

13. Animating `top/left` instead of `transform`

`transform` animations run on the GPU and are buttery smooth. `top` / `left` ones run on the CPU and stutter. Always use transform for movement.

14. Not testing in DevTools' device mode

Phones are most of the internet's traffic. If you've only seen your site at 1440px wide, half your users see broken pages.

— end of chapter twenty —

CHAPTER 21

The complete CSS cheat sheet

Every property and selector you've met in this course, in one place. Tape it to the wall.

Selectors

`*` universal

`p` by tag name

`.class` by class

`#id` by id

`a, b` multiple

`a b` descendant

`a > b` direct child

`a + b` next sibling

`[attr]` has attribute

`[attr="x"]` exact attr

`:hover` hovered

`:focus` focused

`:checked` checked

`:disabled` disabled

`:first-child` first kid

`:nth-child(n)` nth kid

`:not(.x)` not x

`:is(a, b)` any of

`:has(.x)` has child x

`::before` pseudo before

`::after` pseudo after

`::first-letter` drop cap

`::placeholder` input placeholder

`::selection` highlighted text

Color & background

`color`

`background`

`background-color`

`background-image`

`background-size`

`background-position`

`background-repeat`

`opacity`

`color-mix()`

`linear-gradient()`

`radial-gradient()`

`conic-gradient()`

Typography

`font-family``font-size``font-weight``font-style``line-height``letter-spacing``word-spacing``text-align``text-decoration``text-transform``text-wrap``text-shadow``white-space``word-break`

Box model

`width``height``max-width``min-width``max-height``min-height``margin``padding``border``border-radius``box-shadow``box-sizing``aspect-ratio``object-fit`

Display & layout

`display``position``top / right / bottom / left``z-index``overflow``visibility``float``clear`

Flexbox

`display: flex``flex-direction``justify-content``align-items`

`align-content``align-self``flex-wrap``gap``flex``flex-grow``flex-shrink``order`

Grid

`display: grid``grid-template-columns``grid-template-rows``grid-template-areas``grid-area``grid-column``grid-row``gap``justify-items``align-items``place-items``repeat()``minmax()``auto-fit / auto-fill`

Animation & transitions

`transition``transition-property``transition-duration``transition-timing-function``transform``translate()``scale()``rotate()``@keyframes``animation``animation-delay``animation-fill-mode`

Modern

`--custom-property``var()``clamp()``min() / max()``color-mix()``:is() / :where() / :has()``aspect-ratio``backdrop-filter`

`scroll-behavior``container-type`

Media queries

`@media (min-width: ...)``@media (max-width: ...)``@media (hover: hover)``@media (prefers-reduced-motion)``@media (prefers-color-scheme: dark)``@container (min-width: ...)`

— end of chapter twenty-one —

CHAPTER 22

Where to go next

You can now style any webpage you can imagine. The journey from here.

The natural next steps

1. **JavaScript** — interactivity, dynamic content, form validation. (*Lessons 05–06.*)
2. **TypeScript** — JavaScript with safety rails. (*Lesson 07.*)
3. **Tailwind CSS** — utility-first styling. Faster once you know vanilla CSS. (*Lesson 08.*)
4. **A JavaScript framework** — React or Svelte. Components, state, real apps.
5. **CSS architecture** — BEM, CUBE CSS, design tokens, design systems.

Resources worth bookmarking

- **MDN Web Docs** (developer.mozilla.org/CSS) — the official reference. Every property, every value.
- **CSS-Tricks** — great deep-dive articles on every CSS topic.
- **web.dev** — Google's modern CSS articles, always up to date.
- **Can I Use** (caniuse.com) — which browsers support a feature?
- **CodePen** — a playground to test CSS quickly.
- **Coolors** (coolors.co) — generate palettes.
- **Google Fonts** — free, fast, well-maintained typefaces.

Build, build, build

The fastest way to internalize CSS is to copy designs you admire. Pick a real website you like — Bon Appétit, Stripe, Apple — open DevTools and inspect how they do it. Try to recreate one page from scratch. You'll discover gaps in your knowledge instantly, and you'll fill them by reading the docs.

You don't learn CSS by memorizing it. You learn it by trying to build something specific, getting stuck, and looking up the one property you don't know.

Five small projects to try

1. A personal landing page — hero, photo, three sections, footer
2. A pricing page with three plan cards

3. A blog post with proper typography, max-width, drop cap
4. A photo gallery using CSS Grid with one featured image
5. A login form with focus states, hover effects, and validation styles

Each is small enough to finish in an afternoon. Together they cover everything in this book.

— end —

• • •

Tasty Bites · Frontend Cohort